

区块链基础及应用实验报告

Ex6 Tornado-zkSNARK

2213600 刘阳 2210977 陈恩宝

一、实验要求

探索私有事务 (private transactions) 的实现：

- 制作一个简单版本的花费 Tornado 电路；
- 生成赎回 Tornado 的有效性证明。

二、实验过程

1.环境配置

```
> cs251-cash@0.1.0 test
> mocha -s 1s -t 5s test/if_then_else.js test/selective_switch.js test/compute_spend_inputs.js test/spend.js

IfThenElse
  1) should give `false_value` when `condition` = 0
  2) should give `true_value` when `condition` = 1
  3) should enforce that s in {0, 1}

SelectiveSwitch
  4) should not switch when s = 0
  5) should switch when s = 1
  6) should enforce that s in {0, 1}

computeInput
  7) transcript0.txt, depth 0, nullifier 1
  8) transcript1.txt, depth 4, nullifier 4
  9) transcript2.txt, depth 25, nullifier 7

Spend
  ✓ witness computable for depth 0
  ✓ witness computable for depth 1
  ✓ witness computable for depth 2
  10) witness not computable for bad input

3 passing (118ms)
10 failing
```

阅读给定的实验指导书，安装好指导书中给定的库，运行 npm test，并验证是否大多数测试失败，少部分测试通过。由上图可以看出，符合这个要求，说明环境配置成功。

2.了解 circom

首先了解 circom 的语法等内容，同时完成 writeup.md 中的问题。

接着按照 TUTORIAL.md 文件的指导，使用 SmallOddFactorization 电路为 $7 \times 7 \times 17 \times 19 = 2261$ 创建一个证明：

a.编写 circom 电路，并编译将其生成为 json 文件；

b.设置 Snarkjs , 并生成 `proving_key.json` 和 `verification_key.json` 文件将用于生成和验证证明。

c.使用命令查看输入输出格式:

```
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/circuits$ snarkjs info -c circuit.json
# Wires: 32
# Constraints: 23
# Private Inputs: 3
# Public Inputs: 1
# Outputs: 0
```

d.创建input.json文件, 并且计算见证

e.生成见证文件 witness.json;

f.生成证据, 得到 proof.json 与 public.json (公开) ;

g.使用 `snarkjs verify` 命令进行验证, 若返回OK表示证明有效:

```
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/circuits$ snarkjs verify
OK
```

3.开关电路

(1) IfThenElse

具体代码实现如下:

```
template IfThenElse() {
    signal input condition;
    signal input true_value;
    signal input false_value;
    signal output out;

    condition * (1 - condition) === 0;
    if(condition) out <== true_value;
    else out <==false_value;
}
```

根据 condition的值 (0 或 1) 选择输出:

- 如果 `condition` 为 1, 则输出 `true_value`。
- 如果 `condition` 为 0, 则输出 `false_value`。

同时, 约束 `condition * (1 - condition) === 0` 确保 `condition` 必须是布尔值 (即只能为 0 或 1) 。

(2) SelectiveSwitch

具体代码实现如下:

```
template SelectiveSwitch() {
    signal input in0;
    signal input in1;
    signal input s;
    signal output out0;
    signal output out1;

    s * (1 - s) === 0;
```

```

if(s)
{
    out0 <== in1;
    out1 <== in0;
}
else{
    out0 <== in0;
    out1 <== in1;
}
}

```

通过控制信号 `s` 来选择和交换两个输入信号 `in0` 和 `in1` 的位置：

- 如果 `s` 为 1，交换 `in0` 和 `in1`，即 `out0` 和 `out1` 的值交换。
- 如果 `s` 为 0，`out0` 和 `out1` 保持和 `in0`、`in1` 一样的顺序。

同时，约束 `s * (1 - s) == 0` 确保了 `s` 只能是布尔值（0 或 1）。

4.消费电路

按照实验手册要求，该电路应验证所提供的（私有）Merkle 路径是硬币 `H(nullifier, nonce)` 的有效 Merkle 证明，而其根节点的哈希值即为其输入 `digest`。具体实现如下：

```

template Spend(depth) {
    signal input digest; //公开
    signal input nullifier; //公开
    signal private input nonce;
    signal private input sibling[depth];
    signal private input direction[depth];

    component Hash[depth + 1];
    component switches[depth];
    Hash[0] = Mimc2();
    Hash[0].in0 <== nullifier;
    Hash[0].in1 <== nonce;
    for (var i = 0; i < depth; ++i)
    {
        switches[i] = SelectiveSwitch();
        switches[i].in0 <== Hash[i].out;
        switches[i].in1 <== sibling[i];
        switches[i].s <== direction[i];
        Hash[i + 1] = Mimc2();
        // 左节点
        Hash[i + 1].in0 <== switches[i].out0;
        // 右节点
        Hash[i + 1].in1 <== switches[i].out1;
    }
    // 验证是否匹配
    Hash[depth].out == digest;
}

```

a.用 `nullifier` 和 `nonce` 生成硬币的初始哈希值。

b.通过 `sibling` 和 `direction` 数组，逐层计算 Merkle 树中的每个节点的哈希值。

c.最终，验证根节点的哈希值是否与给定的 `digest` 匹配，以证明该支出是有效的。

这个过程利用了 Merkle 树的结构和零知识证明的特点，保证了即使不透露具体的 `sibling` 和 `nonce`，也可以验证支出是否合法。

5.计算花费电路的输入

具体实现如下：

```
function computeInput(depth, transcript, nullifier)
{
  // TODO
  let merkleTree = new SparseMerkleTree(depth);
  for (let i = 0; i < transcript.length; i++)
  {
    const item = transcript[i];
    if (item.length === 1)
      merkleTree.insert(item[0]);
    else
    {
      const nullifierHash = mimc2(item[0], item[1]);
      merkleTree.insert(nullifierHash);
      if (item[0] === nullifier)
        nonce = item[1];
    }
  }
  let spendInput =
  {
    digest: merkleTree.digest,
    nonce: nonce,
    nullifier: nullifier
  };
  let path = merkleTree.path(mimc2(nullifier, nonce));
  for (let i = 0; i < path.length; i++)
  {
    const item = path[i];
    spendInput["sibling[" + i + "]"] = String(item[0]);
    spendInput["direction[" + i + "]"] = String(item[1] ? 1 : 0);
  }
  return spendInput;
}
```

构建 Merkle 树：

- `computeInput` 函数接收三个参数：`depth`（Merkle 树的深度）、`transcript`（记录，包含了历史交易信息）、`nullifier`（用于标识硬币的唯一标识符）。
- 函数首先创建一个空的 **Sparse Merkle Tree**，其深度由 `depth` 参数指定。

插入树的节点：

- 对于每个 `transcript` 项目，执行不同的处理：
 - 如果项目只有一个元素（`item.length === 1`），则将该元素插入 Merkle 树。
 - 如果项目有两个元素（`item.length === 2`），则计算 `nullifier` 和 `nonce` 的哈希（使用 `mimc2` 哈希函数），然后插入 Merkle 树。此时，如果 `item[0]` 是 `nullifier`，则将 `item[1]` 赋值给 `nonce`。

生成支出输入：

- 完成 Merkle 树的插入操作后，构造一个 `spendInput` 对象，包含：
 - `digest`：Merkle 树的根哈希，表示该树的唯一标识。
 - `nonce`：用于验证该硬币是否已被支出的随机数。
 - `nullifier`：硬币的唯一标识符。

计算 Merkle 路径：

- 使用 `merkleTree.path()` 方法计算出从 `nullifier` 和 `nonce` 哈希值到树根的路径（即，Merkle 树的路径），路径包含树中所有节点的哈希值以及每个节点的方向（左/右）。

构造最终的支出输入数据：

- 将每一层的路径信息（兄弟节点和方向）加入到 `spendInput` 对象中。
- `sibling[i]` 存储当前路径上兄弟节点的哈希值。
- `direction[i]` 存储当前路径上走的方向（1 表示右，0 表示左）。

输出支出输入数据：

- 最终返回 `spendInput` 对象，它将包含零知识证明所需的输入数据（`digest`，`nonce`，`nullifier`，以及 Merkle 树路径信息）。

6. 赎回证明

按照实验手册说明，使用 `circom` 和 `snarkjs` 创建一个 SNARK 用来证明深度为 10 的 Merkle 树中存在与 `transcript3.txt` 相对应的 `nullifier`“10137284576094”。并将本组的验证密钥放在 `artifacts/verifier_key_spend.json` 中，将本组的证明放在 `artifacts/proof_spend.json` 中。

具体详见附件 `artifacts/verifier_key_spend.json` 和 `artifacts/proof_spend.json`。

三、实验结果

（一）了解 `circom`

使用命令验证证明，使用之前导出的文件 `verify_key.json`、`proof.json` 和 `public.json` 来检查证明是否有效。

```
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/circuits$ snarkjs printconstraints -c circuit.json
[ 1094412143591963761123202872628637544274182200208017171849102093287904247808 -10944121435919637611123202872628637544274182200208017171849102093287904247808main.factors[0] -2m
ain.smallOdd[0].binaryDecomposition.bits[2] -4main.smallOdd[0].binaryDecomposition.bits[3] -8main.smallOdd[0].binaryDecomposition.bits[4] -16main.smallOdd[0].binaryDecomposition.
bits[5] -32main.smallOdd[0].binaryDecomposition.bits[6] -64main.smallOdd[0].binaryDecomposition.bits[7] ] * [ -109441214359196376111232028726286375442741822002080171718491020932
87904247807 +1094412143591963761112320287262863754427418220020801717184910209328790424780810944121435919637611123202872628637544274182200208017171849102093287904247808main.fact
s[0] +22main.smallOdd[0].binaryDecomposition.bits[2] +44main.smallOdd[0].binaryDecomposition.bits[3] +88main.smallOdd[0].binaryDecomposition.bits[4] +1616main.smallOdd[0].binaryD
ecomposition.bits[5] +3232main.smallOdd[0].binaryDecomposition.bits[6] +6464main.smallOdd[0].binaryDecomposition.bits[7] ] - [ ] = 0
[ 1main.smallOdd[0].binaryDecomposition.bits[2] ] * [ 1 -1main.smallOdd[0].binaryDecomposition.bits[2] ] - [ ] = 0
[ 1main.smallOdd[0].binaryDecomposition.bits[3] ] * [ 1 -1main.smallOdd[0].binaryDecomposition.bits[3] ] - [ ] = 0
[ 1main.smallOdd[0].binaryDecomposition.bits[4] ] * [ 1 -1main.smallOdd[0].binaryDecomposition.bits[4] ] - [ ] = 0
[ 1main.smallOdd[0].binaryDecomposition.bits[5] ] * [ 1 -1main.smallOdd[0].binaryDecomposition.bits[5] ] - [ ] = 0
[ 1main.smallOdd[0].binaryDecomposition.bits[6] ] * [ 1 -1main.smallOdd[0].binaryDecomposition.bits[6] ] - [ ] = 0
[ 1main.smallOdd[0].binaryDecomposition.bits[7] ] * [ 1 -1main.smallOdd[0].binaryDecomposition.bits[7] ] - [ ] = 0
[ 10944121435919637611123202872628637544274182200208017171849102093287904247808 -10944121435919637611123202872628637544274182200208017171849102093287904247808main.factors[1] -2m
ain.smallOdd[1].binaryDecomposition.bits[2] -4main.smallOdd[1].binaryDecomposition.bits[3] -8main.smallOdd[1].binaryDecomposition.bits[4] -16main.smallOdd[1].binaryDecomposition.
bits[5] -32main.smallOdd[1].binaryDecomposition.bits[6] -64main.smallOdd[1].binaryDecomposition.bits[7] ] * [ -109441214359196376111232028726286375442741822002080171718491020932
87904247807 +1094412143591963761112320287262863754427418220020801717184910209328790424780810944121435919637611123202872628637544274182200208017171849102093287904247808main.fact
s[1] +22main.smallOdd[1].binaryDecomposition.bits[2] +44main.smallOdd[1].binaryDecomposition.bits[3] +88main.smallOdd[1].binaryDecomposition.bits[4] +1616main.smallOdd[1].binaryD
ecomposition.bits[5] +3232main.smallOdd[1].binaryDecomposition.bits[6] +6464main.smallOdd[1].binaryDecomposition.bits[7] ] - [ ] = 0
[ 1main.smallOdd[1].binaryDecomposition.bits[2] ] * [ 1 -1main.smallOdd[1].binaryDecomposition.bits[2] ] - [ ] = 0
[ 1main.smallOdd[1].binaryDecomposition.bits[3] ] * [ 1 -1main.smallOdd[1].binaryDecomposition.bits[3] ] - [ ] = 0
[ 1main.smallOdd[1].binaryDecomposition.bits[4] ] * [ 1 -1main.smallOdd[1].binaryDecomposition.bits[4] ] - [ ] = 0
[ 1main.smallOdd[1].binaryDecomposition.bits[5] ] * [ 1 -1main.smallOdd[1].binaryDecomposition.bits[5] ] - [ ] = 0
[ 1main.smallOdd[1].binaryDecomposition.bits[6] ] * [ 1 -1main.smallOdd[1].binaryDecomposition.bits[6] ] - [ ] = 0
[ 1main.smallOdd[1].binaryDecomposition.bits[7] ] * [ 1 -1main.smallOdd[1].binaryDecomposition.bits[7] ] - [ ] = 0
[ 10944121435919637611123202872628637544274182200208017171849102093287904247808 -10944121435919637611123202872628637544274182200208017171849102093287904247808main.factors[2] -2m
ain.smallOdd[2].binaryDecomposition.bits[2] -4main.smallOdd[2].binaryDecomposition.bits[3] -8main.smallOdd[2].binaryDecomposition.bits[4] -16main.smallOdd[2].binaryDecomposition.
bits[5] -32main.smallOdd[2].binaryDecomposition.bits[6] -64main.smallOdd[2].binaryDecomposition.bits[7] ] * [ -109441214359196376111232028726286375442741822002080171718491020932
87904247807 +1094412143591963761112320287262863754427418220020801717184910209328790424780810944121435919637611123202872628637544274182200208017171849102093287904247808main.fact
s[2] +22main.smallOdd[2].binaryDecomposition.bits[2] +44main.smallOdd[2].binaryDecomposition.bits[3] +88main.smallOdd[2].binaryDecomposition.bits[4] +1616main.smallOdd[2].binaryD
ecomposition.bits[5] +3232main.smallOdd[2].binaryDecomposition.bits[6] +6464main.smallOdd[2].binaryDecomposition.bits[7] ] - [ ] = 0
[ 1main.smallOdd[2].binaryDecomposition.bits[2] ] * [ 1 -1main.smallOdd[2].binaryDecomposition.bits[2] ] - [ ] = 0
[ 1main.smallOdd[2].binaryDecomposition.bits[3] ] * [ 1 -1main.smallOdd[2].binaryDecomposition.bits[3] ] - [ ] = 0
[ 1main.smallOdd[2].binaryDecomposition.bits[4] ] * [ 1 -1main.smallOdd[2].binaryDecomposition.bits[4] ] - [ ] = 0
[ 1main.smallOdd[2].binaryDecomposition.bits[5] ] * [ 1 -1main.smallOdd[2].binaryDecomposition.bits[5] ] - [ ] = 0
[ 1main.smallOdd[2].binaryDecomposition.bits[6] ] * [ 1 -1main.smallOdd[2].binaryDecomposition.bits[6] ] - [ ] = 0
[ 1main.smallOdd[2].binaryDecomposition.bits[7] ] * [ 1 -1main.smallOdd[2].binaryDecomposition.bits[7] ] - [ ] = 0
[ -1main.factors[0] ] * [ 1main.factors[1] ] - [ -1main.partialProducts[2] ] = 0
[ -1main.partialProducts[2] ] * [ 1main.factors[2] ] - [ -1main.product ] = 0
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/circuits$
```

```
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/circuits$ snarkjs verify
OK
```

如图，命令输出 OK，说明证明有效。

(二) 电路组件测试

按照要求完成开关电路、消费电路及其输入的编写，使用 npm test 命令进行测试。

```
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/test/circuits$ npm test

> cs251-cash@0.1.0 test
> mocha -s 1s -t 5s test/if_then_else.js test/selective_switch.js test/compute_spend_inputs.js test/spend.js

IfThenElse
  ✓ should give `false_value` when `condition` = 0
  ✓ should give `true_value` when `condition` = 1
  ✓ should enforce that s in {0, 1}

SelectiveSwitch
  ✓ should not switch when s = 0
  ✓ should switch when s = 1
  ✓ should enforce that s in {0, 1}

computeInput
  ✓ transcript0.txt, depth 0, nullifier 1
  ✓ transcript1.txt, depth 4, nullifier 4
  ✓ transcript2.txt, depth 25, nullifier 7

Spend
  ✓ witness computable for depth 0
  ✓ witness computable for depth 1
  ✓ witness computable for depth 2 (600ms)
  ✓ witness not computable for bad input (566ms)

13 passing (2s)
```

可以看到测试全部通过，说明函数编写正确。

(三) 赎回证明

按照要求的流程对其验证：

```
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/test/circuits$ circom spend10.circom -o circuit.json
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/test/circuits$ snarkjs setup
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/test/circuits$ snarkjs calculatewitness
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/test/circuits$ snarkjs proof
hesheep@DESKTOP-174B76P:/mnt/d/Study/大三/区块链/Lab/6/Ex6/test/circuits$ snarkjs verify
OK
```

如图，命令输出 OK，说明证明有效。